DID Protocol — Dual Independent Derivation

Mirror Box Orchestrator · Multi-Agent Governance v1.0

Document scope: This protocol governs how Gemini CLI, Claude Code, and Codex derive, review, and reconcile implementations independently before any Tier 2/3 change is committed. It extends Section 2 of AGENTS.md and must be read alongside it.

---

1. Purpose and Philosophy

The DID protocol exists because no single AI agent is trustworthy enough to own a high-stakes implementation unreviewed. It is not a vote. It is not a committee. It is an engineering control — the same reason aviation uses dual flight computers and nuclear plants use independent trip systems.

The blast radius of a change determines the tier of independent verification required. Low blast radius: one agent proposes, human approves, agent implements. High blast radius: two agents derive blind, human reconciles, only then does implementation begin.

The default chain for MBO is:

1. **Gemini CLI** — reads context, proposes implementation, produces diff
2. **Claude Code** — derives independently (blind), produces its own diff
3. **Human Operator** — reconciles the two diffs, issues "go"
4. **Codex** — tiebreaker only if Gemini and Claude cannot be reconciled after one retry

This is not a hierarchy of trust. It is a pipeline with defined roles.

---

2. Tier Routing — When DID Applies

Every task in `projecttracking.md` has a tier. The tier determines the DID path.

Tier	Blast Radius	Examples	DID Required
0	Docs / governance only	Update NEXT_SESSION.md, close a bug	No
1	Single file, low risk	Fix a log message, rename a variable	No
2	Multi-file or behavioral change	New MCP tool handler, pipeline stage change	Yes — Gemini + Claude
3	Architecture, governance, security	mcp-server.js surgery, AGENTS.md rewrite, new auth path	Yes — Gemini + Claude + Codex available

Rule: When in doubt, escalate the tier. A false Tier 3 costs one extra derivation. A missed Tier 2 can corrupt the graph, break the handshake, or introduce a silent security regression.

---

3. The Standard DID Flow (Tier 2 and Tier 3)

Step 1 — Gemini reads context and derives

Gemini reads the preflight doc, relevant SPEC sections via `graph_search`, and the source files to be changed. It produces a unified diff and one-paragraph rationale. It does not share its output with Claude.

Step 2 — Claude derives blind

Claude reads the same source files and the same preflight doc. It derives independently — no Gemini output in its context window. It produces its own unified diff and one-paragraph rationale.

Step 3 — Human Operator reconciles

The human reads both diffs side by side. Identifies agreement zones and conflict zones. Selects or merges the canonical implementation. Issues "go" with the reconciled diff.

Step 4 — Implementing agent applies the reconciled diff

Either Gemini or Claude (human's choice) applies the reconciled diff using `impl_step.sh` for `src/` writes. Runs the validator, produces the audit package.

---

4. The Tiebreaker Path (Codex)

Codex enters only when the two primary derivations produce an unresolvable conflict. This is rare and is a signal that the preflight spec was underspecified.

Tiebreaker triggers:

- Gemini and Claude make fundamentally different architectural choices
- The human cannot determine which approach is safer without a third data point
- Claude's one retry still does not converge with Gemini

How to invoke Codex as tiebreaker:

```
codex "Tiebreaker derivation — Task 1.1-HXX.
Read: /Users/johnserious/mbo/.dev/preflight/[preflight-doc].md
Conflict zone: [paste only the conflicted diff section from both agents]
Derive your implementation of the conflicted section.
Unified diff only. Do not explain both sides. Pick one."
```

Codex produces its own diff blind. The human picks the majority position (2-of-3) or the approach that best satisfies the preflight acceptance criteria. Codex does not implement — it only breaks ties.

---

5. Claude's Retry Right

If Gemini and Claude disagree, Claude gets one retry before Codex is called.

The human presents Claude with the specific conflict zone only — not Gemini's full diff. Claude may revise its position on that zone. If Claude agrees with Gemini, DID is resolved. If Claude holds its position, Codex is called.

Claude may revise: logic, approach, order of operations, edge case handling.

Claude may not do: change its position simply because Gemini said so. If Claude revises, it must explain why the new position is technically correct — not just that it defers to the other agent. Social capitulation is not a valid derivation.

---

6. Running Each Agent Inside the Other's Shell

All three tools run in the same terminal environment. Any agent can invoke any other.

Gemini running Claude:

```
claude "Blind DID derivation — Task 1.1-HXX.
Read: /Users/johnserious/mbo/.dev/preflight/[preflight].md
Read: [source files listed in preflight]
Produce a unified diff. One-paragraph rationale. Diffs only.
Do not read any Gemini output. Do not ask for clarification."
```

Claude running Gemini:

```
gemini "Blind DID derivation — Task 1.1-HXX.
Read: /Users/johnserious/mbo/.dev/preflight/[preflight].md
Produce a unified diff. One-paragraph rationale. Output diffs only."
```

Either running Codex (tiebreaker):

```
codex "Tiebreaker — Task 1.1-HXX.
Read: /Users/johnserious/mbo/.dev/preflight/[preflight].md
Conflict zone: [paste only the conflicted section from both diffs]
Derive your implementation. Unified diff only."
```

---

7. Output Format Requirements

Every DID derivation produces exactly two artifacts.

Artifact 1 — Unified diff:

```
--- a/src/graph/mcp-server.js
+++ b/src/graph/mcp-server.js
@@ -42,7 +42,4 @@
-  const port = await selectPort();
-  writeManifest({ port, status: 'starting' });
+  const port = 7337;
   httpServer.listen(port, '127.0.0.1', () => {
```

Artifact 2 — One-paragraph rationale:

Example: "Removed dynamic port selection and manifest write because the preflight spec eliminates these entirely — launchd owns the lifecycle. Fixed 7337 matches the plist ProgramArguments. Manifest infrastructure removed because resolve-manifest.js is being deleted and no downstream consumer will exist."

No more than one paragraph. No bullet points. No architecture lecture. What changed, and why does it satisfy the preflight acceptance criteria?

---

8. Blind Derivation Rules

These rules are enforced by the human Operator. Violating them defeats the control.

1. Agent B never sees Agent A's output before producing its own
2. Agents must not ask each other for guidance during derivation
3. The human must not summarize Agent A's approach to Agent B before derivation
4. The retry exception is narrow — Claude may see the specific conflict zone only, not the full Gemini diff

---

9. Reconciliation Heuristics

Conflict type	Resolution heuristic
One agent removes more code	Prefer the smaller diff if preflight says to delete
Different error handling	Prefer hard-fail over silent recovery (MBO policy)
Different ordering	Prefer the order in the preflight's implementation section
Naming disagreement	Prefer the name already used in SPEC or AGENTS.md
One agent adds something not in preflight	Reject unless human explicitly endorses it
Both agents agree	Accept — convergence under blind derivation is strong evidence

Convergence is signal. When Gemini and Claude independently choose the same approach, that approach is almost certainly correct. Divergence is also signal — if they disagree on a fundamental approach, the preflight spec was probably underspecified. Consider updating the preflight before accepting either diff.

---

10. Governance Integration

After every completed DID pass:

- **BUGS.md** — mark the bug COMPLETED with the DID date
- **projecttracking.md** — mark the task COMPLETED
- **CHANGELOG.md** — `fix(scope): description — DID: Gemini/Claude [date]`
- **NEXT_SESSION.md** — update to the next open task

Commit message format:

```
fix(mcp-server): remove lifecycle management, bind fixed port 7337

DID: Gemini (A) + Claude (B), reconciled by human Operator
Implements: Task 1.1-H23
Closes: BUG-069
```

---

11. Escalation Path Summary

```
Task arrives → Tier determination
```

```
Tier 0/1 → Single agent proposes → Human approves → Implement

Tier 2/3 → Gemini derives (A)
              ↓
         Claude derives blind (B)
              ↓
         Human reconciles
              ↓
    ┌── Agree ──────────────────┐
    │                           ↓
    │                       Implement
    │
    └── Conflict → Claude retries on conflict zone (1x)
                        ↓
              ┌── Resolves ──────┐
              │                  ↓
              │              Implement
              │
              └── Still conflicts → Codex tiebreaker
                                        ↓
                                 Human picks majority
                                        ↓
                                    Implement
```

Last updated: 2026-03-13 — Initial version, written for Task 1.1-H14 Tracked in: `.dev/governance/projecttracking.md` — Task 1.1-H14 COMPLETED Stored at: `.dev/governance/DID-PROTOCOL.pdf`